

Design-Weighted Latent Class Analysis of Survey Data: An Adaptable Analysis Script

Openness to government negotiations as a worked example

Kevin

2026-06-17

1 Purpose and how to use this script

This is a complete, adaptable analysis for finding latent classes (hidden respondent types) in categorical survey items collected under a complex sampling design, estimating the class structure so that it describes the **population** rather than the achieved sample, attaching design-based confidence intervals to every estimated probability, writing class membership back to the respondents, and profiling that membership on demographics.

The worked example identifies types of respondents by their **openness to their government negotiating with a neighboring country**, from a multistage sample of about 1,200 respondents with base weights. Everything that is specific to a survey lives in one configuration block (Section 3). To apply this to your own data, edit that block and nothing else.

The script makes four methodological commitments, each explained where it occurs:

1. **The design is respected during estimation**, not only afterward. Class prevalences and item-response probabilities are estimated by *pseudo-maximum likelihood* with the base weights, which is the analogue of declaring `svyset psu [pweight = w], strata(stratum)` in Stata.
2. **poLCA is used as an unweighted benchmark**, because it cannot use design weights, and the gap between it and the weighted fit measures how informative the design is for this typology. A built-in equivalence check confirms the weighted estimator reproduces poLCA exactly when all weights are set to one.
3. **Uncertainty is design-based**. Confidence intervals on every conditional and inclusion probability come from the stratified jackknife (JKn) using replicate weights, which respects the stratification and clustering of a multistage design.

4. **“Don’t know” is treated as a substantive response category**, not as missing data, on the argument that for an attitudinal item like openness to negotiations, declining to take a side is itself a position. The script notes where to change this if your items call for it.

The code uses the tidyverse, contains no `for` or `while` loops (iteration is done with `purrr` and matrix algebra), and is written to fail loudly and informatively rather than silently on edge cases.

2 Setup

```
library(tidyverse)      # data manipulation and plotting
library(matrixStats)    # numerically stable row-wise log-sum-exp and
  ↪ cumulative sums
library(survey)         # complex-design objects, replicate weights, svyglm
library(poLCA)          # unweighted benchmark latent class model
library(clue)           # solve_LSAP: optimal label alignment across fits
library(nnet)           # multinom: multinomial profiling (ships with base R)

set.seed(2026)
`%|||%` <- rlang::`%|||%` # null-coalescing helper used in the EM

# poLCA attaches MASS, whose select() and some matrixStats verbs (count) mask
# their tidyverse equivalents. We namespace-qualify dplyr::select(),
  ↪ dplyr::count(),
# and dplyr::distinct() throughout. This is a deliberate, defensive choice.
```

3 Configuration: the only block you edit to reuse this script

Everything design-specific is gathered here. The worked example below the block generates synthetic data that matches this configuration; when you supply your own data, delete the simulation (Section 5) and point `cfg$data` at your data frame.

```
cfg <- list(
  # --- columns in YOUR data
  ↪ -----
  items = paste0("q", 1:8), # the categorical indicator variables for the
  ↪ LCA
  aux   = c("age_grp", "education", "region"), # demographics for
  ↪ profiling
```

```

strata = "stratum",      # stratum identifier (design)
psu     = "psu",         # primary sampling unit / cluster identifier
  ↪ (design)
weight  = "w",           # base weight (design)

# --- analysis choices
  ↪ -----
K_range = 1:6,           # candidate numbers of latent classes to
  ↪ compare
K_override = NULL,       # set to an integer in K_range to force K*
  ↪ (parsimony /
                                # interpretability override of the BIC
                                ↪ minimum); NULL = use BIC
n_starts = 20L,          # random starts per fit (guards against local
  ↪ maxima)
ref_class = 1L,          # reference class for the multinomial profile
  ↪ (1 = least open)

# --- data
  ↪ -----
data = NULL              # set to your data frame; left NULL to use
  ↪ the simulation
)

```

A note on the **items coding requirement**, which the script checks: each indicator must be a small set of consecutive integer codes starting at 1 (for example 1,2,3,4 for a four-point item). “Don’t know” is treated as an ordinary response category and, by the convention this script assumes throughout, is the **last** code on each item (for example 5 on a four-point item). It is not treated as missing data; every respondent has a complete record and is classified. The check in Section 6 stops with a clear message if an item is mis-coded, because the latent class likelihood indexes response-probability matrices by these integers.

```

# A clean, colorblind-safe plotting theme reused by every figure.
theme_lca <- function(base_size = 11) {
  theme_minimal(base_size = base_size) +
    theme(panel.grid.minor = element_blank(),
          panel.grid.major.x = element_blank(),
          strip.text = element_text(face = "bold", size = rel(0.85)),
          plot.title = element_blank(),
          legend.position = "bottom",
          legend.title = element_text(face = "bold", size = rel(0.85)),
          plot.caption = element_text(hjust = 0, size = rel(0.78), color =
            ↪ "grey30"))
}

```

```

}
wu_pal <- setNames(viridisLite::viridis(2, begin = 0.2, end = 0.75),
  c("Weighted (population)", "Unweighted (poLCA)"))

```

4 The latent class model

A latent class model supposes that each respondent belongs to one of K unobserved classes, and that **within a class the items are statistically independent** (the local-independence assumption). Writing π_k for the share of the population in class k , and ρ_{jck} for the probability that a member of class k gives response c to item j , the probability of observing respondent i 's full pattern of answers is a mixture over the classes:

$$P(\mathbf{y}_i) = \sum_{k=1}^K \pi_k \prod_{j=1}^J \rho_{j y_{ij} k}, \quad \text{with } \sum_{c=1}^{C_j} \rho_{jck} = 1 \text{ for every item } j \text{ and class } k.$$

The π_k are the **inclusion (class-size) probabilities** and the ρ_{jck} are the **conditional (item-response) probabilities**; together they are the *measurement model*. Each item is treated as an unordered (nominal) categorical variable, which is exactly what **poLCA** assumes, so the benchmark in Section 11 compares like with like. Treating “Don’t know” as a category simply adds one response level c to the items where it appears.

4.1 Why ordinary poLCA is not enough

poLCA maximizes the *unweighted* likelihood, so it estimates the class structure of the **achieved sample**. Under a complex design with unequal selection probabilities, the sample over- or under-represents parts of the population, and the unweighted estimates inherit that distortion. To describe the **population**, each respondent’s contribution to the likelihood must be weighted by their base weight w_i , giving the *design-weighted pseudo-log-likelihood*

$$\ell_w(\theta) = \sum_i w_i \log \left(\sum_{k=1}^K \pi_k \prod_j \rho_{j y_{ij} k} \right).$$

Maximizing ℓ_w yields *pseudo-maximum-likelihood* estimates of the finite-population parameters. This is the standard design-based device, and it is what we implement below, because poLCA has no argument for weights.

5 Worked-example data (delete this section to use your own data)

This section simulates a multistage sample that matches the configuration: 9 strata, about 20 primary sampling units in total, roughly 1,200 respondents, base weights that vary across strata, and a design that is **informative** because the more heavily weighted strata are also the ones least open to negotiation. Eight items measure openness on four-point scales, with a fifth “Don’t know” level present on every item for 5 to 10 percent of respondents. Because the data are generated from known parameters, Sections 11 and 12 can show how closely the weighted estimator recovers the truth.

The simulation itself avoids loops: respondents are laid out with `tidyr::expand_grid`, and class draws and item draws are vectorized through inverse-CDF samplers.

```
# ---- generative truth
↪ -----
K_true <- 3L
J       <- 8L
n_resp_levels <- 4L           # substantive responses 1..4
dk_code <- n_resp_levels + 1L # "Don't know" coded as 5

# Class-conditional response profiles for the 4 substantive levels:
# Class 1 "Open" leans high, Class 2 "Ambivalent" central, Class 3 "Closed"
↪ leans low.
prof <- list(
  open      = c(0.08, 0.17, 0.35, 0.40),
  ambivalent = c(0.20, 0.32, 0.30, 0.18),
  closed     = c(0.45, 0.30, 0.17, 0.08)
)
true_rho4 <- list(open = prof$open, ambivalent = prof$ambivalent, closed =
↪ prof$closed)

# ---- multistage design
↪ -----
# 9 strata; PSUs per stratum chosen to total ~20; 60 respondents per PSU.
psu_per_stratum <- c(2L, 2L, 2L, 2L, 2L, 2L, 2L, 3L, 3L) # sums to 20 PSUs
resp_per_psu    <- 60L
H <- length(psu_per_stratum)

# Stratum-level openness mix (alpha) and base weight, made to covary =>
↪ informative.
grade <- seq(0, 1, length.out = H)
alpha_h <- cbind(open      = 0.20 + 0.45 * grade,
                  ambivalent = 0.30,
```

```

        closed      = 0.50 - 0.45 * grade)
alpha_h <- alpha_h / rowSums(alpha_h)
w_stratum <- round(seq(2.2, 0.6, length.out = H), 2) # open strata sampled
  ↪ more (lower weight)

# Build the respondent frame without loops.
frame <- tibble(stratum = rep(seq_len(H), times = psu_per_stratum)) |>
  mutate(psu_in_stratum = sequence(psu_per_stratum)) |>
  mutate(psu = row_number()) |> # unique
  ↪ PSU id
  tidyr::uncount(resp_per_psu, .id = "within") |>
  mutate(id = row_number(),
         w = w_stratum[stratum])

n <- nrow(frame)

# ---- draw latent class for each respondent (vectorized inverse-CDF)
  ↪ -----
draw_rows <- function(P) {
  cum <- matrixStats::rowCumsums(P)
  u <- runif(nrow(P))
  pmin(rowSums(cum < u) + 1L, ncol(P))
}
true_class <- draw_rows(alpha_h[frame$stratum, , drop = FALSE])

# ---- draw item responses given class, then overlay "Don't know"
  ↪ -----
rho_by_class <- list(true_rho4$open, true_rho4$ambivalent, true_rho4$closed)

draw_one_item <- function(seed_offset) {
  set.seed(900 + seed_offset)
  P <- do.call(rbind, rho_by_class[true_class]) # n x 4 response probs per
  ↪ respondent
  resp <- draw_rows(P)
  # Don't-know overlay: 5-10% per item, mildly higher among the ambivalent
  ↪ class.
  dk_p <- 0.05 + 0.03 * (true_class == 2)
  is_dk <- rbinom(length(resp), 1, dk_p) == 1
  resp[is_dk] <- dk_code
  resp
}
item_mat <- map(seq_len(J), draw_one_item) |> set_names(paste0("q",
  ↪ seq_len(J)))

```

```

sim <- frame |>
  bind_cols(as_tibble(item_mat)) |>
  mutate(true_class = true_class,
         age_grp    = factor(sample(c("18-34", "35-54", "55+"), n, TRUE)),
         education  = factor(sample(c("HS or less", "Some college", "BA+"), n,
                                   TRUE)),
         # region carries a mild signal so a profiling covariate is non-null
         region     = factor(if_else(runif(n) < plogis(-0.3 +
                                   0.6*(true_class==1)),
                               "Coastal", "Interior")))

# Population truth for later comparison.
pop_pi_true <- as.numeric(prop.table(xtabs(w ~ true_class, data = sim)))

# Hand the simulated data to the pipeline.
cfg$data <- sim

```

6 Preparing and validating the data

Before any modeling, the script confirms the indicators are well coded and the design columns are present, stopping with an explicit message if anything is wrong. This is the robustness layer: it converts silent downstream errors into clear, early ones.

“Don’t know” requires no special handling here, by design: it is simply one of the response categories (the last code on each item), so it enters the model exactly like any other answer. Every respondent has a complete record and is classified. The only preparation step is to confirm each item is coded as consecutive integers starting at 1, recoding if there are gaps, because the latent class likelihood indexes response-probability matrices by these integer codes.

```

# Pull the working data and confirm required columns exist.
dat <- cfg$data
stopifnot(is.data.frame(dat))
required_cols <- c(cfg$items, cfg$aux, cfg$strata, cfg$psu, cfg$weight)
missing_cols <- setdiff(required_cols, names(dat))
if (length(missing_cols) > 0)
  stop("These configured columns are not in the data: ", paste(missing_cols,
    collapse = ", "))

# Each item must be consecutive integers starting at 1 (DK is just the last
  code).

```

```

# Recode to close any gaps, so the likelihood can index response-probability
  ↪ matrices.
recode_consecutive <- function(x) as.integer(factor(x, levels =
  ↪ sort(unique(x))))
dat_prepared <- dat |> mutate(across(all_of(cfg$items), ~
  ↪ recode_consecutive(as.integer(.x))))

cats <- map_int(cfg$items, ~ max(dat_prepared[[.x]]))
names(cats) <- cfg$items
J_items <- length(cfg$items)

# Report the category counts so the analyst can confirm the coding (including
  ↪ the DK level).
tibble(item = cfg$items, categories = cats) |> knitr::kable(align = "lr")

```

item	categories
q1	5
q2	5
q3	5
q4	5
q5	5
q6	5
q7	5
q8	5

7 The weighted EM estimator

The script estimates the measurement model with a weighted Expectation-Maximization (EM) algorithm. EM alternates two steps until the weighted log-likelihood stops improving.

The **E-step** computes each respondent's posterior probability of belonging to each class, given the current parameters:

$$p_{ik} = \frac{\pi_k \prod_j \rho_{j y_{ij} k}}{\sum_m \pi_m \prod_j \rho_{j y_{ij} m}}.$$

The **M-step** updates the parameters as *weighted* proportions, which is the only place the base weights enter:

$$\hat{\pi}_k = \frac{\sum_i w_i p_{ik}}{\sum_i w_i}, \quad \hat{\rho}_{jck} = \frac{\sum_i w_i p_{ik} \mathbf{1}(y_{ij} = c)}{\sum_i w_i p_{ik}}.$$

Because the objective being maximized is the weighted *pseudo*-log-likelihood rather than an ordinary likelihood, these are *pseudo*-maximum-likelihood estimates. They are consistent for the population parameters, but, as Section 10 explains, their standard errors cannot be read off the curvature of the objective in the usual way and must instead come from replication.

Two implementation choices make this robust and loop-free. First, the E-step is computed on the log scale and normalized with the **log-sum-exp** identity, which prevents numerical underflow when many small probabilities are multiplied together. Second, the iteration is expressed as a `purrr::reduce fold` over iteration indices rather than a `while` loop: each step takes the current parameter state and returns the updated one, and the fold stops once the weighted log-likelihood stops improving.

```
# Random starting values: a random class distribution and random
  ↪ response-prob matrices.
rand_init <- function(cats, K) {
  list(pi = { x <- runif(K); x / sum(x) },
       rho = map(cats, function(Cj) {
         m <- matrix(runif(Cj * K) + 0.1, Cj, K); sweep(m, 2, colSums(m),
  ↪  "/" )
         })))
}

# One full EM run, written as a fold (no loops). Y is a list of integer item
  ↪ vectors;
# OH is a list of one-hot indicator matrices (one per item).
em_run <- function(Y, OH, cats, w, K, init = NULL, maxit = 800L, tol = 1e-8)
  ↪ {
  nn <- length(Y[[1]])
  st0 <- c(init %||% rand_init(cats, K),
           list(post = NULL, ll = -Inf, iter = 0L, done = FALSE))

  step <- function(state, .iter) {
    if (isTRUE(state$done)) return(state)
    # E-step on the log scale. log(rho_j)[y, ] picks each respondent's
    ↪ answered-category
    # log-probability for item j; summed across items and added to the log
    ↪ class prior.
    log_terms <- map2(state$rho, Y, ~ log(.x)[.y, , drop = FALSE])
    logdens <- reduce(log_terms, `+`) + matrix(log(state$pi), nn, K, byrow =
  ↪ TRUE)
  }
```

```

lse <- matrixStats::rowLogSumExps(logdens)
post <- exp(logdens - lse)
ll <- sum(w * lse)
# M-step: weighted proportions. crossprod(OH_j, w*post) sums weighted
  ↳ responsibilities
# over respondents who gave each category; columns renormalized to valid
  ↳ probabilities.
wp <- w * post
den <- colSums(wp)
rho_n <- map(OH, function(oh) {
  num <- pmax(crossprod(oh, wp), 1e-12)
  sweep(num, 2, colSums(num), "/")
})
list(pi = den / sum(den), rho = rho_n, post = post, ll = ll,
     iter = state$iter + 1L,
     done = abs(ll - state$ll) < tol * (abs(state$ll) + 1))
}
out <- reduce(seq_len(maxit), step, .init = st0)
out$converged <- out$done
out
}

# Build integer item vectors and one-hot category-indicator matrices (one per
  ↳ item).
make_inputs <- function(df, items, cats) {
  Y <- map(items, ~ as.integer(df[[.x]]))
  OH <- map2(items, cats, ~ (outer(as.integer(df[[.x]]), seq_len(.y), `==`))
  ↳ + 0)
  list(Y = Y, OH = OH)
}

# Class labels are arbitrary; align any fit to a reference by matching
  ↳ response profiles
# with the Hungarian algorithm, so classes are comparable across starts,
  ↳ fits, and replicates.
profiles_of <- function(rho) do.call(cbind, map(rho, t)) # K x sum(Cj)
align_to <- function(fit, ref) {
  K <- length(fit$pi)
  Pf <- profiles_of(fit$rho); Pr <- profiles_of(ref$rho)
  cost <- outer(seq_len(K), seq_len(K),
                Vectorize(function(a, b) sum((Pf[a, ] - Pr[b, ])^2)))
  asg <- clue::solve_LSAP(cost)

```

```

inv <- integer(K); inv[as.integer(asg)] <- seq_len(K)
list(pi = fit$pi[inv],
     rho = map(fit$rho, ~ .x[, inv, drop = FALSE]),
     post = if (!is.null(fit$post)) fit$post[, inv, drop = FALSE] else
       ↪ NULL,
     ll = fit$ll, converged = fit$converged %||% NA)
}

# Fit at a given K from many random starts; keep the best weighted
↪ log-likelihood.
fit_lca <- function(df, w, cats, items, K, starts, ref = NULL) {
  inp <- make_inputs(df, items, cats)
  cands <- map(seq_len(starts), function(s) {
    set.seed(1000 + s + 17 * K)
    em_run(inp$Y, inp$OH, cats, w, K)
  })
  best <- cands[[which.max(map_dbl(cands, "ll"))]]
  if (!is.null(ref)) best <- align_to(best, ref)
  best
}

```

8 Choosing the number of classes

The number of classes K is not known in advance and is chosen by comparing a sequence of models. Three quantities guide that choice, and because they are routinely misread, each is explained here in the terms you would use with an analyst.

BIC (Bayesian Information Criterion) scores a model by its fit, penalized for the number of parameters it uses:

$$\text{BIC} = -2\hat{\ell} + \text{df} \cdot \ln n, \quad \text{df} = (K - 1) + K \sum_j (C_j - 1).$$

The first term rewards fit (a higher log-likelihood $\hat{\ell}$ makes it smaller), and the second term charges a price for every additional parameter, a price that grows with the sample size n . **Lower BIC is better.** You read the column down the candidate values of K and look for the **minimum**; often the values fall steeply, reach a low point, and rise again, and that low point is the recommended number of classes. The interpretation is “the number of classes that explains the response patterns well without adding classes that only fit noise.” BIC is the primary tool for this decision in latent class analysis.

AIC (Akaike Information Criterion) has the same form with a smaller, fixed penalty (2 per parameter instead of $\ln n$):

$$\text{AIC} = -2\hat{\ell} + 2 \text{df}.$$

It is reported for completeness, but because its penalty is lighter it tends to **favor too many classes**, so when AIC and BIC disagree, prefer BIC for a parsimonious, interpretable solution.

Relative entropy answers a different question. BIC and AIC ask “how many classes?”; entropy asks “once we have them, how cleanly are respondents sorted into them?”

$$E = 1 - \frac{-\sum_i \sum_k p_{ik} \ln p_{ik}}{n \ln K}.$$

It runs from 0 to 1. A value near 1 means most respondents have a posterior probability close to 1 for a single class, so the classification is sharp; a value near 0 means respondents are spread across classes with no clear home. A common rule of thumb treats values above about 0.8 as good separation. Entropy is **not** a measure of fit and must **not** be used to choose K on its own: a two-class model almost always has higher entropy than a five-class model simply because fewer classes are easier to separate. Read it alongside BIC as a description of how usable the resulting classification is, not as a selection criterion.

All enumeration here uses the **weighted** fit, so the chosen K describes the population, not the achieved sample. The script reports all three quantities across the candidate range and selects the K that minimizes the weighted BIC.

```
df_k <- function(K, cats) (K - 1) + K * sum(cats - 1)

entropy_R2 <- function(post, K) {
  if (K == 1) return(NA_real_)
  1 - (-sum(post * log(pmax(post, 1e-12)))) / (nrow(post) * log(K))
}

w_vec <- dat_prepared[[cfg$weight]]

enum <- map_dfr(cfg$K_range, function(K) {
  fit <- fit_lca(dat_prepared, w_vec, cats, cfg$items, K, cfg$n_starts)
  tibble(K = K,
    logLik = fit$ll,
    df = df_k(K, cats),
    AIC = -2 * fit$ll + 2 * df_k(K, cats),
    BIC = -2 * fit$ll + df_k(K, cats) * log(nrow(dat_prepared)),
    entropy = entropy_R2(fit$post, K),
    converged = fit$converged)
})

K_bic <- enum |> slice_min(BIC, n = 1) |> pull(K) |> first()
# Optional manual override of the class count: force K* when the BIC minimum
↪ is not
```

```

# the most parsimonious or interpretable solution. Must be one of the
↪ enumerated K.
if (!is.null(cfg$K_override)) {
  stopifnot(as.integer(cfg$K_override) %in% cfg$K_range)
  K_star <- as.integer(cfg$K_override)
} else {
  K_star <- K_bic
}
if (K_star < 2L)
  stop("K* < 2: a single class has no latent structure to profile. Inspect
↪ enumeration or set cfg$K_override >= 2.")

enum |>
  mutate(across(c(logLik, AIC, BIC), ~ round(.x, 1)),
          entropy = round(entropy, 3)) |>
  knitr::kable(align = "r",
               caption = "Weighted fit statistics by number of classes. K* is
↪ the minimum-BIC solution unless cfg$K_override is set.")

```

Table 2: Weighted fit statistics by number of classes. K* is the minimum-BIC solution unless `cfg$K_override` is set.

K	logLik	df	AIC	BIC	entropy	converged
1	-19501.5	32	39067.1	39230.0	NA	TRUE
2	-18989.1	65	38108.2	38439.0	0.655	TRUE
3	-18931.4	98	38058.7	38557.5	0.744	TRUE
4	-18873.5	131	38009.1	38675.9	0.748	TRUE
5	-18821.9	164	37971.9	38806.7	0.787	TRUE
6	-18779.3	197	37952.6	38955.4	0.780	TRUE

The weighted BIC is minimized at **2 classes**, and the analysis proceeds with **2 classes** (these coincide unless `cfg$K_override` is set to impose a more parsimonious or interpretable solution). The figure shows BIC and entropy across the candidate range; BIC is read for selection and entropy for separation.

```

enum |>
  dplyr::select(K, BIC, entropy) |>
  pivot_longer(-K, names_to = "criterion", values_to = "value") |>
  mutate(criterion = recode(criterion, BIC = "Weighted BIC", entropy =
↪ "Relative entropy")) |>

```

```
ggplot(aes(K, value)) +
  geom_line(linewidth = 0.6, color = viridisLite::viridis(1, begin = 0.4)) +
  geom_point(size = 2.4, color = viridisLite::viridis(1, begin = 0.4)) +
  facet_wrap(~ criterion, scales = "free_y") +
  scale_x_continuous(breaks = cfg$K_range) +
  labs(x = "Number of latent classes (K)", y = NULL) +
  theme_lca()
```

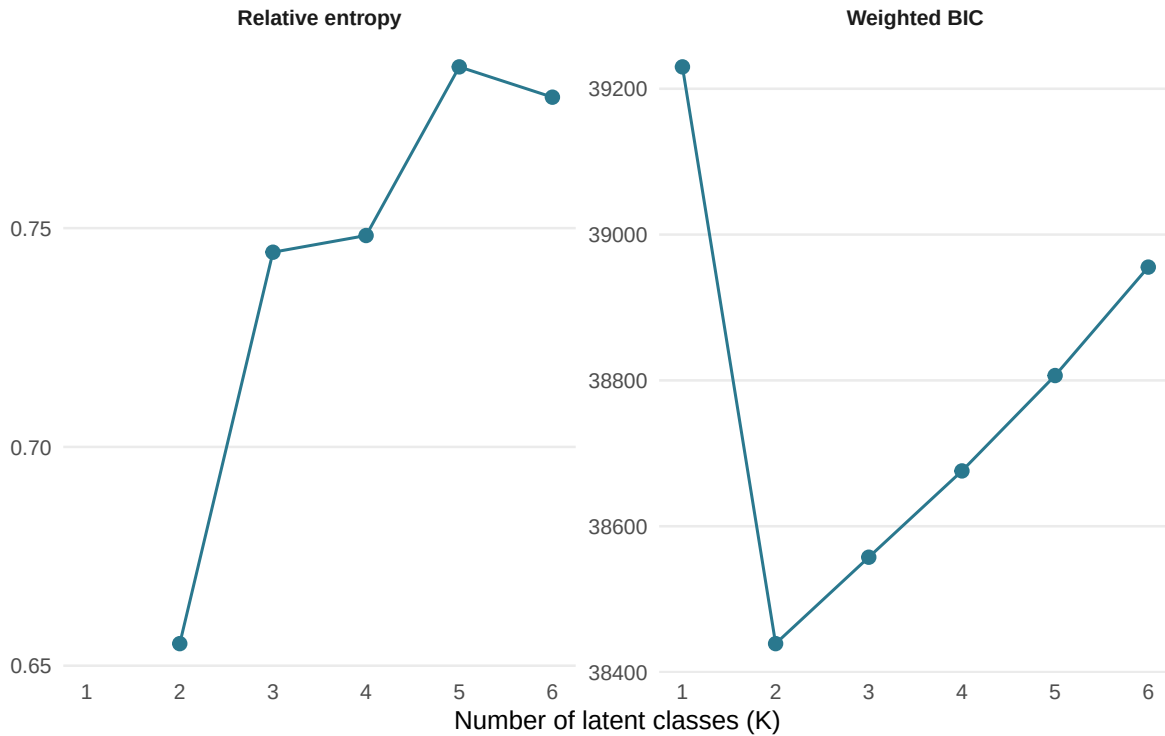


Figure 1: Weighted BIC and relative entropy across candidate numbers of classes. The chosen K minimizes BIC; entropy describes how cleanly respondents are classified.

9 The population measurement model

At the selected K , the script fits the model two ways on the same data: **weighted** (the population measurement model) and **unweighted** (which, fit with **poLCA**'s assumptions, is what **poLCA** itself targets). The comparison in Section 11 then shows what the design weights change.

The model has two sets of parameters, and both deserve a plain reading because they are the substantive output of the analysis.

The **inclusion probabilities** π_k are the **class prevalences**: the fraction of the population in each class. They sum to 1 across classes. A prevalence of 0.35 for a class means an estimated 35 percent of the population belongs to that type. These are the headline numbers, the size of each segment.

The **conditional probabilities** ρ_{jck} are **item-response probabilities**: given that a respondent is in class k , the probability they give answer c to item j . For each class and item these sum to 1 across the answer categories. They are how you *characterize and name* a class. If, in one class, the probability of the most-open answer is high on every negotiation item, that class is the “open” type; if another class places most of its probability on “don’t know” across items, that is an “ambivalent / no-opinion” type. Reading down the conditional probabilities for a class is reading its profile.

Ordering the classes for interpretation. Latent classes have no natural order; the algorithm could label them in any sequence, which makes comparison and plotting harder to follow. The script therefore relabels them along the substantive dimension of interest, openness, so that Class 1 is the least open and the highest-numbered class is the most open. The ordering score is each class’s expected response on the substantive answer categories, *excluding the “don’t know” category*, because “don’t know” is not a point on the openness scale: a class full of DK responses is not “more open” than one full of opposed responses, it is simply uncommitted. DK is therefore set aside when ordering and reported separately as each class’s DK propensity. This relabeling changes only the class numbers, never the model. It does assume the substantive answer codes run from least to most open; if your items are coded in the other direction, reverse the score.

```
# --- fit at K_star, weighted and unweighted
↪ -----
fit_w_raw <- fit_lca(dat_prepared, w_vec, cats, cfg$items, K_star,
↪   cfg$n_starts)

# --- order classes by substantive openness, EXCLUDING the DK (last) category
↪ --
# For each class, average over items the expected response code among
↪   substantive
# answers (the DK row is dropped and the remaining probabilities
↪   renormalized).
# Low score = leans toward the low (least open) end; high score = most open.
openness_score <- map_dbl(seq_len(K_star), function(k) {
  mean(map_dbl(fit_w_raw$rho, function(rho_j) {
    sub <- rho_j[-nrow(rho_j), k]      # drop the DK (last) category
    sub <- sub / sum(sub)              # renormalize over substantive answers
```

```

    sum(seq_along(sub) * sub)          # expected substantive code
  )))
})
ord <- order(openness_score)          # ascending: Class 1 = least open

reorder_classes <- function(fit, ord) {
  list(pi = fit$pi[ord],
       rho = map(fit$rho, ~ .x[, ord, drop = FALSE]),
       post = if (!is.null(fit$post)) fit$post[, ord, drop = FALSE] else
         ↪ NULL,
       ll = fit$ll, converged = fit$converged %||% NA)
}

# Weighted fit becomes the reference label order; the unweighted polCA
  ↪ benchmark and
# every jackknife replicate are aligned to it later (Sections 10-11), so
  ↪ "Class 1"
# means the same thing in every table and figure.
fit_w <- reorder_classes(fit_w_raw, ord)

# DK propensity per class: weighted-model probability of the DK (last)
  ↪ category,
# averaged over items. Reported separately because DK is excluded from
  ↪ ordering.
dk_propensity <- map_dbl(seq_len(K_star), function(k)
  mean(map_dbl(fit_w$rho, ~ .x[nrow(.x), k])))

# --- write posteriors and modal class onto the data NOW
  ↪ -----
# Doing this here lets the same replicate design (built next) serve both the
# measurement-model confidence intervals and the profiling regression.
post_w <- fit_w$post
colnames(post_w) <- paste0("post_class", seq_len(K_star))
modal_int <- max.col(post_w, ties.method = "first")
dat_prepared <- dat_prepared |>
  bind_cols(as_tibble(post_w)) |>
  mutate(modal_class = factor(modal_int, levels = seq_len(K_star)),
         max_posterior = post_w[cbind(seq_len(n()), modal_int)])

```


9.1 Checking the local-independence assumption: design-weighted bivariate residuals

Every quantity above rests on the assumption stated in Section 4: within a class, the items are independent. BIC and the entropy describe *relative* fit and separation, but neither asks whether that local-independence assumption actually holds at the selected number of classes, and if it is violated and left unmodeled the class structure and the prevalences can be biased (Visser & Depaoli, 2022, *Structural Equation Modeling* 29(6):971–982). This subsection checks it directly.

The tool is the **bivariate residual** (BVR): for each pair of items it compares the association present in the data with the association the fitted model predicts under local independence, and reports what is left over (Asparouhov & Muthén, 2015, *Structural Equation Modeling* 22(2):169–177). For items a and b , local independence makes the joint response probability a class-weighted product of the two item-response probabilities,

$$\hat{M}_{cc'} = \sum_{k=1}^K \hat{\pi}_k \hat{\rho}_{ack} \hat{\rho}_{bc'k},$$

the probability of answering c to item a and c' to item b if the only thing the two items share is the latent class. The design enters exactly as in the M-step: the observed joint proportion $p_{cc'}$ is the **base-weighted** share of respondents giving that pair of answers, so it describes the population, not the achieved sample. The bivariate residual, placed on the sample-size scale, is the Pearson distance between the two,

$$\text{BVR}_{ab} = n \sum_c \sum_{c'} \frac{(p_{cc'} - \hat{M}_{cc'})^2}{\hat{M}_{cc'}}, \quad \text{df} = (C_a - 1)(C_b - 1),$$

where C_a and C_b are the numbers of response categories (including “don’t know”). It is the residual association the latent classes do not account for: small when the pair is conditionally independent given class, large when the two items remain tied together after the classes are removed.

How to read it. Under simple random sampling and local independence, BVR/df has an expected value of about one (it behaves like a chi-square over its degrees of freedom), so values near one are unremarkable and values well above one flag a pair the model leaves correlated. There is **no universal cutoff**; the recommended practice is to scan the pairs, attend to the largest, and keep the number of comparisons in mind rather than testing each pair at a fixed level (Visser & Depaoli, 2022). Two cautions specific to this design are stated rather than hidden. First, the chi-square reference is approximate even under simple random sampling, because it ignores the covariance among the expected cell counts; a parametric bootstrap p-value is the more honest reference, and under a complex design that bootstrap must resample

primary sampling units within strata to respect the clustering. Second, clustering inflates the sampling variability of $p_{cc'}$ relative to a simple random sample, so the effective null spread is wider than the chi-square implies, and modestly elevated values should be read against the design effect, not against the simple-random-sample expectation of one. The statistic below is therefore a **screen** for local dependence, computed with the survey weights so the associations are population associations, not a calibrated test.

What to do when residuals are large. The standard remedy is to relax local independence only where the data demand it, adding a direct effect between the offending pair and refitting, one pair at a time, until no large residuals remain (Visser & Depaoli, 2022). In this worked example the weighted BIC selected two classes against a three-class truth (Section 12), so the model is deliberately parsimonious and under-extracted, and under-extraction is itself a common source of elevated bivariate residuals: structure a third class would have absorbed reappears as residual association between items. Read this subsection alongside enumeration: persistent large residuals at the chosen number of classes signal that the parsimonious solution is leaving systematic structure on the table, and the choice is then between a direct effect, an extra class, or accepting the simpler model with its limitation stated.

```
# Local independence says items are uncorrelated WITHIN a class. For items a,
↪ b the
# model-implied joint probability under that assumption is
#   M[c, c'] = sum_k pi_k * rho_a[c, k] * rho_b[c', k] = rho_a %*% (pi *
↪ t(rho_b)),
# written without diag() to avoid the length-one diag() trap (valid for any
↪ K). The
# observed joint proportion is base-weighted, the same population weighting
↪ as the
# M-step. BVR_ab = n * sum (p_obs - M)^2 / M, df = (C_a - 1)(C_b - 1). The
↪ pair loop
# is a purrr fold over all item pairs (no for/while).
bvr_pair <- function(a, b) {
  rho_a <- fit_w$rho[[a]]
  rho_b <- fit_w$rho[[b]]
  M      <- pmax(rho_a %*% (fit_w$pi * t(rho_b)), 1e-12)      # C_a x C_b
↪ model-implied
  ya     <- factor(dat_prepared[[cfg$items[a]]], levels =
↪ seq_len(nrow(rho_a)))
  yb     <- factor(dat_prepared[[cfg$items[b]]], levels =
↪ seq_len(nrow(rho_b)))
  tab    <- tibble(.wt = dat_prepared[[cfg$weight]], ya = ya, yb = yb)
  obs    <- as.matrix(xtabs(.wt ~ ya + yb, data = tab))      # weighted
↪ joint counts
  p_obs  <- obs / sum(obs)
```

```

bvr    <- nrow(dat_prepared) * sum((p_obs - M)^2 / M)
df     <- (nrow(rho_a) - 1L) * (nrow(rho_b) - 1L)
tibble(item_a = cfg$items[a], item_b = cfg$items[b],
        BVR = bvr, df = df, ratio = bvr / df)
}

bvr_tbl <- combn(seq_along(cfg$items), 2, simplify = FALSE) |>
  map_dfr(~ bvr_pair(.x[1], .x[2]))

# Heatmap of BVR/df over item pairs (one tile per pair; ~1 unremarkable, >>1
  ↪ flags
# residual dependence). Mirrors the rho heatmap's styling.
bvr_grid <- bvr_tbl |>
  mutate(item_a = factor(item_a, levels = cfg$items),
         item_b = factor(item_b, levels = cfg$items),
         lbl_col = if_else(ratio > max(ratio) * 0.6, "white", "grey15"))

ggplot(bvr_grid, aes(item_a, fct_rev(item_b), fill = ratio)) +
  geom_tile(color = "white", linewidth = 0.5) +
  geom_text(aes(label = sprintf("%.1f", ratio), color = lbl_col), size = 2.6)
  ↪ +
  scale_color_identity() +
  scale_fill_viridis_c(name = "BVR / df") +
  labs(x = NULL, y = NULL) +
  theme_lca() +
  theme(panel.grid = element_blank())

```

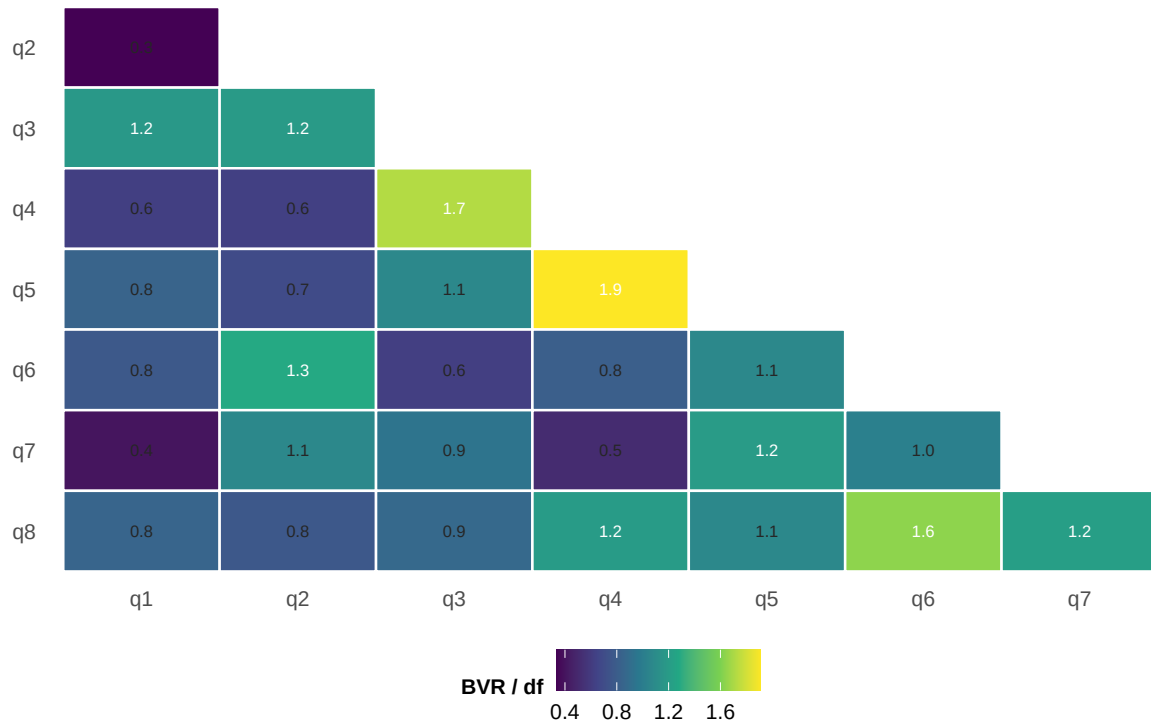


Figure 2: Design-weighted bivariate residuals (BVR/df) for every item pair. Under local independence and simple random sampling each value is near 1; values well above 1 flag residual association the latent classes do not explain. The reference is approximate under a clustered design.

The largest design-weighted bivariate residuals, the pairs to inspect first:

```
bvr_tbl |>
  slice_max(ratio, n = 6) |>
  transmute(`Item pair` = paste(item_a, item_b, sep = " - "),
            BVR = round(BVR, 1), df = df, `BVR / df` = round(ratio, 2)) |>
  knitr::kable(align = "lrrr",
               caption = paste("Largest design-weighted bivariate
                               ↪ residuals.",
                               "Under local independence and simple random
                               ↪ sampling",
                               "BVR/df is approximately 1; there is no
                               ↪ universal cutoff,",
                               "so the largest values are inspected with",
                               "multiple-comparison awareness, and the
                               ↪ reference is",
```

```
"approximate under a clustered design.))
```

Table 3: Largest design-weighted bivariate residuals. Under local independence and simple random sampling BVR/df is approximately 1; there is no universal cutoff, so the largest values are inspected with multiple-comparison awareness, and the reference is approximate under a clustered design.

Item pair	BVR	df	BVR / df
q4 - q5	30.4	16	1.90
q3 - q4	27.4	16	1.71
q6 - q8	26.1	16	1.63
q2 - q6	20.5	16	1.28
q7 - q8	19.4	16	1.21
q4 - q8	19.1	16	1.20

10 Design-based confidence intervals via the stratified jackknife

10.1 Why the survey weights alone are not enough

A natural question at this point is: the data already came with weights, so why not simply use them and be done? The answer is that survey weights do **one** of the two jobs that honest estimation requires, and the confidence intervals depend on the other.

The weights do the first job, **point estimation**: they make the prevalences and response probabilities describe the *population* rather than the achieved sample. That is precisely the weighted M-step in Section 7, $\hat{\pi}_k = \sum_i w_i p_{ik} / \sum_i w_i$, and the script does exactly that. If the only goal were the estimates themselves, “just use the weights” would be the complete answer.

The weights do **not** do the second job, **variance estimation**, because the precision of a survey estimate depends on *how the sample was selected*, not only on the weights. Two surveys can share an identical weight column yet differ greatly in precision: a sample of 1,200 people drawn as 20 clusters carries far less information than 1,200 people drawn independently, because respondents within a cluster tend to resemble one another, so each additional person in a cluster adds less new information than an independent person would. The weight attached to a respondent says nothing about which cluster they came from; the **design** (the stratum and PSU identifiers) is what carries that information. This is why the code below declares `svydesign(ids = ~psu, strata = ~stratum, weights = ~w)` and builds replicate weights from it, rather than working from the weight column alone.

The practical consequence of ignoring this is concrete and one-directional. If you weighted the data and then computed a standard error as though the weighted sample were a simple random sample of independent people, the estimate would be correct but the standard error would typically be **too small**, often by a factor of 1.5 to 2.5 in clustered samples. That inflation factor is the *design effect*. Using the weights for the estimate while ignoring the design for the variance, the single most common error in survey analysis, produces confidence intervals that are falsely narrow and overstates how much the data actually tell you.

There is a second, independent reason the variance cannot come from the weighted fit alone. The weighted objective $\sum_i w_i \log(\cdot)$ is a *pseudo*-log-likelihood, not a genuine likelihood for any probability model, because fractional and unequally weighted contributions do not correspond to a count of independent observations. The standard maximum-likelihood machinery that would ordinarily hand back standard errors from the curvature of the likelihood (the information matrix) therefore does not apply. The variance has to be estimated by *replication*, refitting the model on systematically perturbed versions of the design, which is what the stratified jackknife does and what the next subsection describes.

In one sentence for a non-technical audience: **the weights tell us who the population is, and the design tells us how sure we are**, and we need both.

10.2 Estimating the design-based variance

Confidence intervals must reflect the sampling design, not pretend the data are a simple random sample. The script uses **replicate weights**: the design is declared, converted to a set of stratified jackknife (JKn) replicates, and the entire weighted EM is re-estimated on each replicate. The spread of the replicate estimates gives the design-based variance,

$$\widehat{\text{Var}}(\hat{\theta}) = \sum_r c_r (\hat{\theta}^{(r)} - \hat{\theta})^2,$$

where each JKn replicate r deletes one primary sampling unit and reweights the rest of its stratum, with the scale factor c_r that **survey** supplies. With about 20 PSUs across 9 strata this produces roughly 20 replicates and about 11 degrees of freedom for the variance, which is the true precision of the design regardless of the 1,200 respondents; intervals will be honestly wide.

Two robustness points are built in. The design declaration uses **nest = TRUE** so PSUs nested within strata are handled correctly. And **lonely PSUs** (a stratum with a single PSU, which would break JKn) are handled by setting **options(survey.lonely.psu = "adjust")**, which centers such a stratum's contribution at the grand mean rather than erroring; the script reports whether any were encountered. Each replicate refit is **warm-started at the full-sample solution and aligned to it**, which keeps the 20 refits stable, fast, and free of label switching.

```

options(survey.lonely.psu = "adjust")

# Report PSU structure so the analyst sees the degrees of freedom they
  ↪ actually have.
# Note: dplyr::count() is namespace-qualified because matrixStats (loaded for
# rowLogSumExps) also exports a count(), and it loads after dplyr here.
psu_structure <- dat_prepared |>
  dplyr::distinct(.data[[cfg$strata]], .data[[cfg$psu]]) |>
  dplyr::count(.data[[cfg$strata]], name = "psus_in_stratum")
n_psu_total <- nrow(dplyr::distinct(dat_prepared, .data[[cfg$psu]]))
n_strata <- n_distinct(dat_prepared[[cfg$strata]])
lonely_any <- any(psu_structure$psus_in_stratum < 2)

# One declaration of the design as stratified JKn replicates, reused for both
  ↪ the
# measurement-model CIs (here) and the multinomial profile (Section 13), so
  ↪ the two
# rest on identical replicate weights. If your survey instead SHIPS replicate
  ↪ weights,
# replace the body of this helper with a single
#   svrepdesign(repweights = ..., weights = ..., type = ...)
# and nothing downstream changes.
make_rep_design <- function(data) {
  svydesign(ids = reformulate(cfg$psu), strata = reformulate(cfg$strata),
    weights = reformulate(cfg$weight), data = data, nest = TRUE) |>
    as.svrepdesign(type = "JKn")
}
rep_des <- make_rep_design(dat_prepared)

# Parameter bookkeeping as a TIBBLE, in the exact order flatten_fit()
  ↪ produces.
# This carries item/category/class as real columns, so nothing is recovered
  ↪ from
# string parsing later and the survey-question names are preserved
  ↪ downstream.
param_meta <- bind_rows(
  tibble(kind = "pi", item = NA_character_, category = NA_integer_, class =
    ↪ seq_len(K_star)),
  imap_dfr(cats, function(Cj, item_name)
    expand_grid(class = seq_len(K_star), category = seq_len(Cj)) |> #
    ↪ category varies fastest
    transmute(kind = "rho", item = item_name, category, class))

```

```

)
param_names <- param_meta |>
  mutate(nm = if_else(kind == "pi",
    sprintf("pi[%d]", class),
    sprintf("rho[%s,cat%d,class%d]", item, category,
      ↪ class))) |>
  pull(nm)
flatten_fit <- function(fit) set_names(c(fit$pi, unlist(map(fit$rho,
  ↪ as.vector)))), param_names)

# The statistic survey::withReplicates evaluates on each replicate: refit the
  ↪ weighted EM
# on the replicate weights, warm-started at the full-sample fit and aligned
  ↪ to it.
theta_fun <- function(w_rep, data) {
  inp <- make_inputs(data, cfg$items, cats)
  fit <- em_run(inp$Y, inp$OH, cats, w_rep, K_star,
    init = fit_w[c("pi", "rho")], maxit = 400L)
  flatten_fit(aligned_to(fit, fit_w))
}

rep_var <- withReplicates(rep_des, theta_fun)
est_vec <- flatten_fit(fit_w)
se_vec <- sqrt(diag(vcov(rep_var)))

# Design-based 95% CIs use the t critical value on the design degrees of
  ↪ freedom
# (number of PSUs minus number of strata for JKn), not the normal 1.96; this
  ↪ matches
# survey's own confint() and matters when the design carries few df.
crit_meas <- qt(0.975, survey::degf(rep_des))

# Tidy table of every probability with its design-based 95% CI:
  ↪ item/category/class
# are columns, ready to filter, join, and plot.
ci_tbl <- param_meta |>
  mutate(estimate = as.numeric(est_vec), se = as.numeric(se_vec),
    lo = pmax(0, estimate - crit_meas * se),
    hi = pmin(1, estimate + crit_meas * se))

```

The class-size (inclusion) probabilities, with their design-based intervals and each class's propensity to answer “don't know”:


```

ci_tbl |>
  filter(kind == "pi") |>
  transmute(Class = paste("Class", class),
            Prevalence = round(estimate, 3),
            SE = round(se, 4),
            `95% CI` = sprintf("%.3f, %.3f", lo, hi),
            `DK propensity` = round(dk_propensity, 3)) |>
  knitr::kable(align = "lrrrr",
               caption = "Population class prevalences (inclusion
               ↪ probabilities) with design-based JKn confidence intervals.
               ↪ Classes are ordered from least to most open; DK propensity
               ↪ is the average probability of a don't-know response and is
               ↪ reported separately because it does not enter the
               ↪ ordering.")

```

Table 4: Population class prevalences (inclusion probabilities) with design-based JK_n confidence intervals. Classes are ordered from least to most open; DK propensity is the average probability of a don't-know response and is reported separately because it does not enter the ordering.

Class	Prevalence	SE	95% CI	DK propensity
Class 1	0.439	0.0228	[0.388, 0.489]	0.066
Class 2	0.561	0.0228	[0.511, 0.612]	0.056

The item-response (conditional) probabilities are shown as a heatmap, which is far easier to scan than overlaid lines when there are several items: each panel is a class, each row an item, each column a response category, and the shade (and printed value) is the probability of that answer within that class. Reading a panel top to bottom gives that class's profile; comparing panels shows how the classes differ. The design-based confidence interval for every cell is in the exported table `lca_measurement_model_with_CIs.csv`; intervals are omitted from the heatmap only to keep it legible.

```

rho_plot <- ci_tbl |>
  filter(kind == "rho") |>
  mutate(Class = factor(paste("Class", class), levels = paste("Class",
  ↪ seq_len(K_star))),
         item = factor(item, levels = cfg$items),
         category = factor(category),
         lbl = sprintf("%.2f", estimate),
         lbl_col = if_else(estimate > 0.55, "white", "grey15"))

```

```
ggplot(rho_plot, aes(category, fct_rev(item), fill = estimate)) +
  geom_tile(color = "white", linewidth = 0.5) +
  geom_text(aes(label = lbl, color = lbl_col), size = 2.4) +
  scale_color_identity() +
  facet_wrap(~ Class, nrow = 1) +
  scale_fill_viridis_c(limits = c(0, 1), name = "P(response | class)") +
  labs(x = "Response category (last column = Don't know)", y = NULL) +
  theme_lca() +
  theme(panel.grid = element_blank(),
        legend.key.width = unit(1.4, "cm"))
```

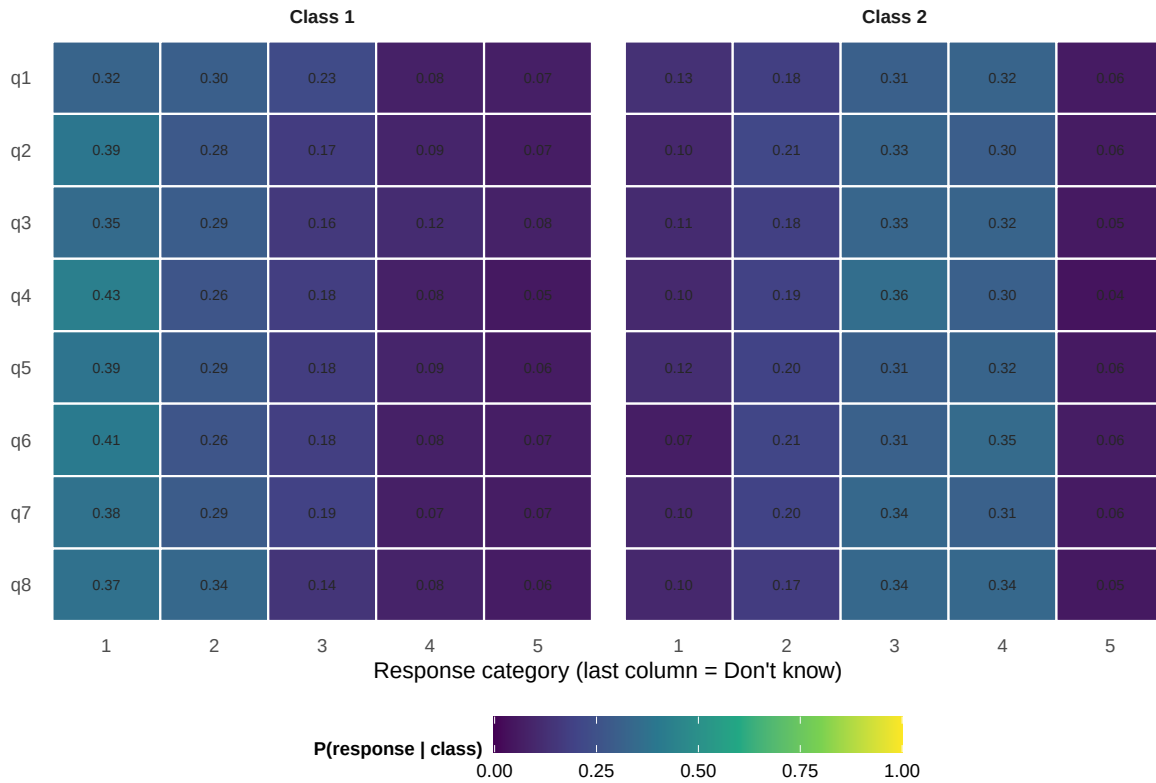


Figure 3: Population item-response (conditional) probabilities by class. Each panel is a class (ordered least to most open); rows are items, columns are response categories (the last column is Don't know). Cell shade and label give $P(\text{response} | \text{class})$. This is the measurement model that defines the respondent types.

11 Benchmark against poLCA, and a validation check

Two comparisons with poLCA close the loop. First, a **validation**: when all weights are set to one, the weighted EM and poLCA maximize the identical likelihood, so their estimates must agree up to label order and convergence tolerance. Confirming this is the strongest evidence that the custom EM is implemented correctly. Second, a **substantive benchmark**: poLCA fit normally (unweighted) estimates the achieved-sample structure, and its gap from the weighted estimates measures how much the design matters for this typology. The script also reports the K that poLCA's own BIC would select, since informative sampling can change the apparent number of classes.

```
# poLCA needs a data frame of items coded 1..C with no NA (it uses na.rm).
polca_df <- dat_prepared |> dplyr::select(all_of(cfg$items)) |>
  ↪ as.data.frame()
polca_f <- as.formula(paste0("cbind(", paste(cfg$items, collapse = ", "), ")
  ↪ ~ 1"))

# --- substantive benchmark: poLCA at K_star, unweighted ---
set.seed(7)
pl <- poLCA(polca_f, data = polca_df, nclass = K_star, nrep = 10,
  ↪ maxiter = 5000, na.rm = TRUE, verbose = FALSE)
pl_aligned <- align_to(list(pi = as.numeric(pl$P),
  ↪ rho = map(pl$probs, t),
  ↪ post = pl$posterior), fit_w)

# --- poLCA's own BIC-selected K across the candidate range ---
# poLCA is not defined for a single class, so the unweighted benchmark scans
  ↪ K >= 2.
polca_K_range <- cfg$K_range[cfg$K_range >= 2L]
polca_bic <- map_dfr(polca_K_range, function(K) {
  set.seed(10 + K)
  m <- poLCA(polca_f, data = polca_df, nclass = K, nrep = 5,
    ↪ maxiter = 5000, na.rm = TRUE, verbose = FALSE)
  tibble(K = K, poLCA_BIC = m$bic)
})
polca_K <- polca_bic |> slice_min(poLCA_BIC, n = 1) |> pull(K) |> first()

# --- validation: weighted EM with all weights = 1 should match poLCA ---
fit_unit <- align_to(fit_lca(dat_prepared, rep(1, nrow(dat_prepared)), cats,
  ↪ cfg$items,
  ↪ K_star, cfg$n_starts), fit_w)
max_pi_gap <- max(abs(fit_unit$pi - pl_aligned$pi))
max_rho_gap <- max(map2_dbl(fit_unit$rho, pl_aligned$rho, ~ max(abs(.x -
  ↪ .y))))
```

```

tibble(`Latent class` = paste("Class", seq_len(K_star)),
      `Population (weighted)` = round(fit_w$pi, 3),
      `Sample (poLCA)` = round(pl_aligned$pi, 3),
      `Difference (pp)` = round((fit_w$pi - pl_aligned$pi) * 100, 1)) |>
knitr::kable(align = "lrrr",
             caption = "Class prevalence: weighted (population) versus
↪ unweighted poLCA (achieved sample). The difference is the
↪ imprint of the design.")

```

Table 5: Class prevalence: weighted (population) versus unweighted poLCA (achieved sample). The difference is the imprint of the design.

Latent class	Population (weighted)	Sample (poLCA)	Difference (pp)
Class 1	0.439	0.402	3.7
Class 2	0.561	0.598	-3.7

The validation check returns a maximum class-size discrepancy of 5×10^{-4} and a maximum response-probability discrepancy of 3×10^{-4} between the unit-weight EM and poLCA; values near zero confirm the EM is correct. The weighted BIC selected 2 classes, and poLCA's unweighted BIC selected 2.

```

bind_rows(
  tibble(Class = paste("Class", seq_len(K_star)), prevalence = fit_w$pi,
        Estimator = "Weighted (population)"),
  tibble(Class = paste("Class", seq_len(K_star)), prevalence = pl_aligned$pi,
        Estimator = "Unweighted (poLCA)")
) |>
ggplot(aes(Class, prevalence, fill = Estimator)) +
geom_col(position = position_dodge(width = 0.7), width = 0.6) +
scale_fill_manual(values = wu_pal) +
scale_y_continuous(limits = c(0, NA)) +
labs(x = NULL, y = "Class prevalence", fill = NULL) +
theme_lca()

```

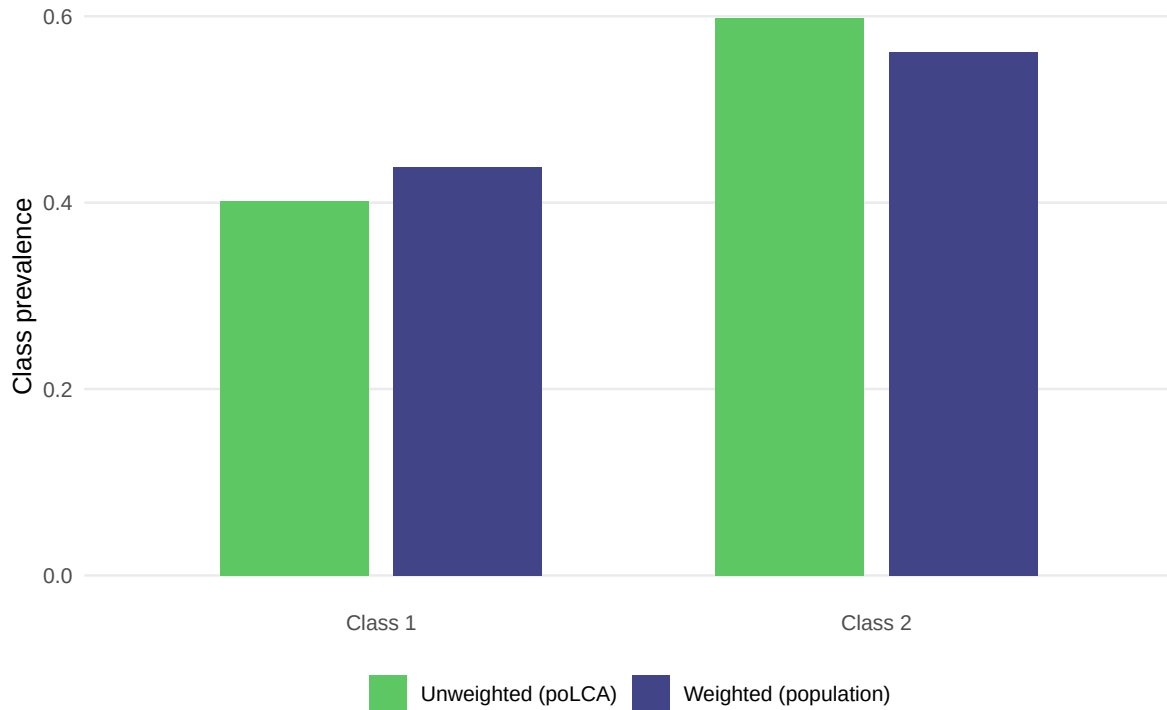


Figure 4: Population (weighted) versus achieved-sample (poLCA) class prevalences. Separation between the two indicates an informative design for this typology.

12 How much the design matters, against known truth

Because the worked example is simulated, the script can show the *gain* from weighting directly: how close each estimator’s prevalences are to the population truth. On real data this section instead reports only the weighted-versus-unweighted divergence (Section 11), because the truth is unknown and no error can be computed without fabricating a target.

This comparison is only well defined when the selected number of classes equals the number used to generate the data. When the weighted BIC selects a different number (a central, weakly separated class can be merged under BIC, for instance), a class-by-class prevalence comparison has no meaning, and the script says so rather than forcing a misleading table. When the counts do match, the estimated classes are first aligned to the *true* generating classes by the weighted overlap of their modal assignments, so each row compares like with like.

```
# Valid only when the selected K equals the generating K. Otherwise report
↪ that
# plainly: a class-by-class comparison across different K is not defined.
```

```

K_match <- (K_star == K_true)

if (K_match) {
  # Align estimated classes to the TRUE classes by the weighted overlap of
  ↪ modal
  # assignment with the known generating class (Hungarian algorithm,
  ↪ maximizing overlap).
  ov_df <- tibble(wt = dat_prepared[[cfg$weight]],
                  est = max.col(fit_w$post, ties.method = "first"),
                  tru = dat_prepared$true_class)
  overlap <- as.matrix(xtabs(wt ~ est + tru, data = ov_df))
  to_true <- as.integer(clue::solve_LSAP(overlap, maximum = TRUE)) #
  ↪ estimated row -> true col
  est_in_true_order <- integer(K_true)
  est_in_true_order[to_true] <- seq_len(K_star) # true
  ↪ col -> estimated row

  truth_tbl <- tibble(
    `Latent class (true)` = paste("Class", seq_len(K_true)),
    `Truth` = round(pop_pi_true, 3),
    `Weighted` = round(fit_w$pi[est_in_true_order], 3),
    `poLCA` = round(pl_aligned$pi[est_in_true_order], 3)) |>
    mutate(`|Weighted - Truth|` = abs(Weighted - Truth),
           `|poLCA - Truth|` = abs(poLCA - Truth))

  err_w <- round(max(truth_tbl$`|Weighted - Truth|`) * 100, 1)
  err_u <- round(max(truth_tbl$`|poLCA - Truth|`) * 100, 1)

  truth_text <- sprintf(
    "the weighted estimator's prevalences fall within %s percentage points of
    ↪ the population truth, against up to %s points for unweighted poLCA",
    err_w, err_u)

  display_tbl <- truth_tbl |>
    mutate(across(c(`|Weighted - Truth|`, `|poLCA - Truth|`), ~ round(.x,
    ↪ 3)))
} else {
  err_w <- err_u <- NA_real_
  truth_text <- sprintf(
    "the weighted BIC selected %d classes while the data were generated with
    ↪ %d, so a class-by-class prevalence comparison is not defined here
    ↪ (this merging or splitting is itself an informative result)",

```

```

    K_star, K_true)
  display_tbl <- tibble(Note = truth_text)
}

knitr::kable(display_tbl, align = "l",
              caption = "Recovery of the known population prevalences
↪ (simulation only).")

```

Table 6: Recovery of the known population prevalences (simulation only).

Note
the weighted BIC selected 2 classes while the data were generated with 3, so a class-by-class prevalence comparison is not defined here (this merging or splitting is itself an informative result)

In this run, the weighted BIC selected 2 classes while the data were generated with 3, so a class-by-class prevalence comparison is not defined here (this merging or splitting is itself an informative result). On your own data, delete this section.

13 Profiling class membership on demographics

The posterior class probabilities and the modal class were written back onto the data in Section 9, so each respondent now carries their most likely type and the full posterior. Per the analysis plan, **no posterior threshold is applied**: every respondent is kept, so the profiling sample is the full design sample and the weights retain their meaning. The task now is to describe *who* falls in each class, in terms of demographics.

13.1 Can a demographic used in the weighting also be a predictor here?

Yes. A variable such as age that was used to construct the survey weights can also appear as a predictor in this regression, and doing so is not double-counting. The weights and the covariate play different roles: the weights make the sample represent the population, and the covariate measures an association *within* that represented population. In fact, including the variables that drive the design as predictors is often the *better* choice. Pfeffermann (1993, *International Statistical Review* 61(2):317–337) and Gelman (2007, *Statistical Science* 22(2):153–164) make the underlying point: when the model conditions on the variables related to selection, the sampling becomes (approximately) ignorable, the weighted and unweighted estimates converge,

and the analysis is less sensitive to the weights. So if age shaped the weights, putting age in the model is a feature, not a violation, and it generally improves efficiency.

Two qualifications. First, interpret the coefficient conditionally: it is the association of age with class membership *holding the other predictors fixed*, not a marginal effect. Second, the design-based standard errors (from the replicate weights below) remain the correct ones whether or not a predictor overlaps with the weighting variables. The circularity that genuinely must be avoided is a different one, raised earlier: a variable used as an **indicator** of the latent classes must not also be a **predictor** of them, because then you are explaining the classes with the very items that defined them. Demographics used in weighting are fine; the survey items that built the typology are not.

13.2 A design-based multinomial model for more than two classes

Because membership has more than two categories, the profile uses a **multinomial logistic regression**: it models the log-odds of each class against a reference class (here Class 1, the least open) as a function of the demographics. This generalizes the two-class logistic to the full set of classes in one model.

Getting design-correct uncertainty for a multinomial under a complex design is the same problem solved for the measurement model, and it is solved the same way. Within each jackknife replicate the coefficients are estimated by weighted multinomial likelihood (`nnet::multinom` with the replicate weights), and `survey::withReplicates` aggregates the spread of those replicate estimates into design-based standard errors, using the **same JK_n replicate weights** built in Section 10. This keeps the whole analysis on one coherent variance footing rather than mixing methods.

The same two cautions apply and are not hidden. The outcome is the *modal* class, an estimated quantity, so regressing it on covariates **attenuates** the coefficients toward zero relative to the true class-covariate association; this is a conservative, descriptive profile, not unbiased inference. For genuinely inferential statements, the bias-adjusted three-step methods (BCH and the maximum-likelihood correction) replace the modal regression while keeping this design-based variance. A simpler alternative, if a single segment is the whole question, is a binary design-based logistic regression (`svyglm` with `family = quasibinomial()`) of that one class against the rest.

```
# Reference class first so nnet::multinom uses it as the baseline.
ref_lvls <- c(cfg$ref_class, setdiff(seq_len(K_star), cfg$ref_class))
dat_prepared <- dat_prepared |>
  mutate(modal_class = factor(as.integer(as.character(modal_class)), levels
    ↪   = ref_lvls))

# Rebuild the design so its data carry the releveled outcome, then JKn
  ↪   replicates,
```



```

# using the same helper as the measurement model (identical variance
  ↪ footing).
prof_rep <- make_rep_design(dat_prepared)
prof_formula <- reformulate(cfg$aux, response = "modal_class")

# Statistic for withReplicates: weighted multinomial coefficients, flattened
  ↪ to a
# named vector (class-major: all terms of class 2, then class 3, ...).
multinom_theta <- function(w_rep, data) {
  data[[".w"]] <- as.numeric(w_rep) # inject so model.frame
  ↪ resolves it from data
  m <- nnet::multinom(prof_formula, data = data, weights = .w, trace =
  ↪ FALSE, maxit = 1000)
  cf <- coef(m)
  if (is.null(dim(cf))) # 2-class outcome: coef()
    ↪ is a named vector
    cf <- matrix(cf, nrow = 1, dimnames = list(m$lev[2], names(cf)))
  set_names(as.vector(t(cf)),
    as.vector(t(outer(rownames(cf), colnames(cf), paste, sep =
    ↪ "::<")))))
}
mn_rep <- withReplicates(prof_rep, multinom_theta)
mn_est <- multinom_theta(weights(prof_rep, type = "sampling"), dat_prepared)
  ↪ # full-sample
mn_se <- sqrt(diag(vcov(mn_rep)))
  ↪ # design SEs
stopifnot(length(mn_est) == length(mn_se))

# Same t-on-design-df critical value as the measurement model, for the OR
  ↪ intervals.
crit_prof <- qt(0.975, survey::degf(prof_rep))

mn_tbl <- tibble(coef = names(mn_est),
  estimate = as.numeric(mn_est),
  se = as.numeric(mn_se)) |>
  separate(coef, into = c("class", "term"), sep = "::<") |>
  filter(term != "(Intercept)") |>
  mutate(comparison = paste0("Class ", class, " vs ", cfg$ref_class),
    OR = exp(estimate),
    lo = exp(estimate - crit_prof * se),
    hi = exp(estimate + crit_prof * se))

```

```
mn_tbl |>
  transmute(Comparison = comparison, Predictor = term,
    `Log-odds` = round(estimate, 3), SE = round(se, 3),
    `Odds ratio` = round(OR, 2),
    `OR 95% CI` = sprintf("[% .2f, % .2f]", lo, hi)) |>
  knitr::kable(align = "llrrrr",
    caption = paste0("Design-based multinomial profile of class
      ↪ membership on demographics, each class against Class ",
        cfg$ref_class, " (least open). Coefficients
      ↪ are descriptive and attenuated by classification error."))
```

Table 7: Design-based multinomial profile of class membership on demographics, each class against Class 1 (least open). Coefficients are descriptive and attenuated by classification error.

Comparison	Predictor	Log-odds	SE	Odds ratio	OR 95% CI
Class 2 vs 1	age_grp35-54	-0.196	0.155	0.82	[0.58, 1.16]
Class 2 vs 1	age_grp55+	-0.129	0.168	0.88	[0.61, 1.27]
Class 2 vs 1	educationHS or less	0.138	0.157	1.15	[0.81, 1.62]
Class 2 vs 1	educationSome college	0.153	0.139	1.17	[0.86, 1.58]
Class 2 vs 1	regionInterior	-0.352	0.128	0.70	[0.53, 0.93]

```
ggplot(mn_tbl, aes(OR, term, color = comparison)) +
  geom_vline(xintercept = 1, linetype = "dashed", color = "grey55") +
  geom_pointrange(aes(xmin = lo, xmax = hi), size = 0.35, linewidth = 0.4,
    position = position_dodge(width = 0.5)) +
  facet_wrap(~ comparison) +
  scale_x_log10() +
  scale_color_manual(values = setNames(viridisLite::viridis(max(1, K_star -
    ↪ 1), begin = 0.1, end = 0.75),
    unique(mn_tbl$comparison)), guide =
    ↪ "none") +
  labs(x = "Odds ratio versus reference class (log scale)", y = NULL) +
  theme_lca()
```

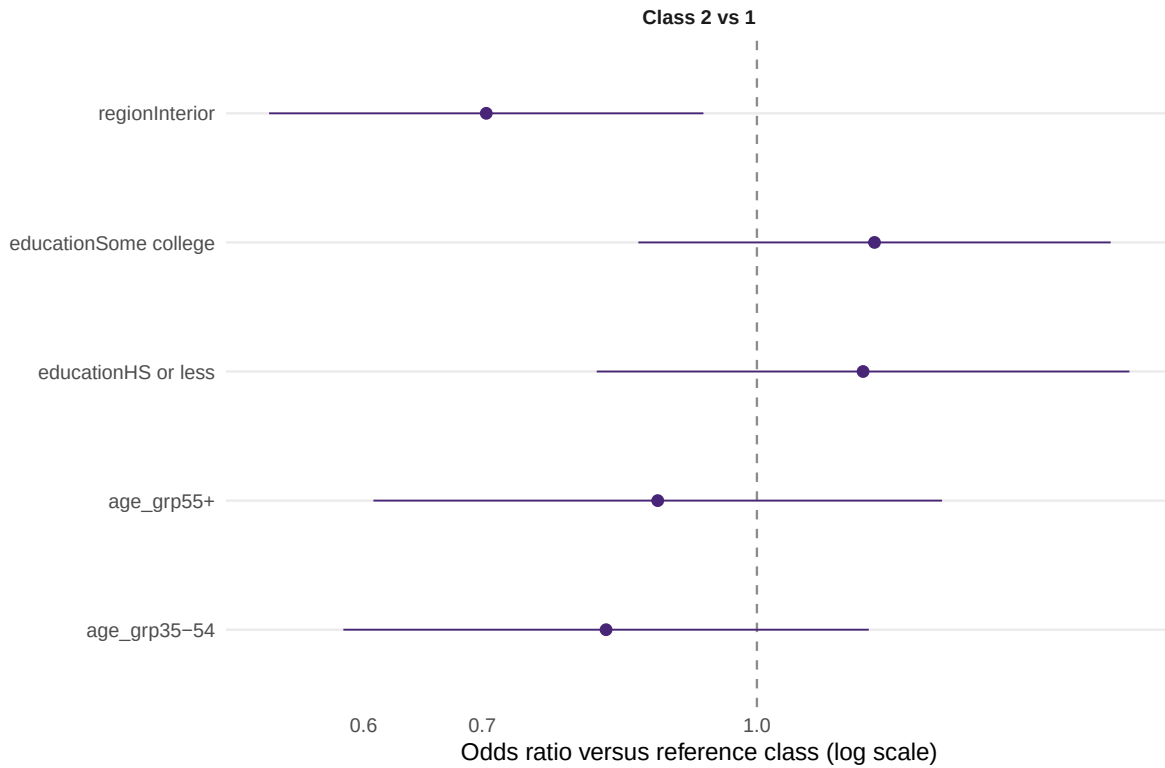


Figure 5: Design-based odds ratios with 95% intervals from the multinomial profile, each non-reference class versus the reference class, by predictor. Points right of 1 indicate higher odds of that class (versus reference) for that demographic level. Read as a descriptive, design-correct profile, attenuated by classification error.

14 Exporting results

```
readr::write_csv(enum, "lca_enumeration.csv")
readr::write_csv(ci_tbl |> mutate(across(where(is.numeric), ~ round(.x, 5))),
  "lca_measurement_model_with_CIs.csv")
readr::write_csv(dat_prepared |> dplyr::select(id, all_of(c(cfg$strata,
  ↪   cfg$psu, cfg$weight))),
  starts_with("post_class"),
  ↪   modal_class,
  ↪   max_posterior),
  "lca_caselevel_assignments.csv")
```

Three files are written: the enumeration table, the full measurement model with design-based confidence intervals (one row per probability, with `item`, `category`, and `class` as columns), and the case-level assignments (posteriors, modal class, and maximum posterior) ready to merge back into the survey data.

15 Adapting this script to your survey: a checklist

1. **Point `cfg$data` at your data frame** and delete Section 5 (the simulation) and Section 12 (the truth comparison), which exist only because the example is simulated.
2. **Set `cfg$items`** to your indicator columns, coded as consecutive integers from 1, with “Don’t know” as the **last** category on each item. The category-count table in Section 6 lets you confirm the coding.
3. **Set `cfg$strata`, `cfg$psu`, `cfg$weight`** to your design columns. This is the analogue of `svyset psu [pweight = w], strata(stratum)`. If your file ships **replicate weights** instead of strata and PSU identifiers, replace each `svydesign(...)` plus `as.svrepdesign(...)` pair with a single `svrepdesign(repweights = ..., weights = ..., type = ...)`; the rest of the variance code is unchanged.
4. **Set `cfg$aux`** to the demographics you want in the profile. Demographics used to build the weights (age, for instance) are fine and often helpful here; the only variables to exclude are the survey items that defined the classes.
5. **Check the openness ordering.** Classes are ordered by their expected response on the substantive categories, excluding DK, assuming the answer codes run from least to most open. If your items are coded in the other direction, reverse the score in Section 9; if “openness” is not your dimension, replace the score with whatever scalar orders your classes meaningfully.
6. **Set `cfg$ref_class`** to the class you want as the multinomial baseline (default is Class 1, the least open after ordering).
7. **Run, then read enumeration first.** Choose K on the weighted BIC, sanity-checked against entropy and interpretability; the validation check should show the EM matching `poLCA` at unit weights before you trust the weighted output.
8. **Mind the degrees of freedom.** Your design-based intervals carry roughly (number of PSUs minus number of strata) degrees of freedom; with few PSUs they will be wide, and that width is honest.
9. **Treat the profile as descriptive.** If you need inferential statements about demographic predictors, move to a bias-adjusted three-step model, keeping the design-based variance.

```
sessionInfo()
```

```
R version 4.6.0 (2026-04-24 ucrt)
```

Platform: x86_64-w64-mingw32/x64
Running under: Windows 11 x64 (build 26200)

Matrix products: default
LAPACK version 3.12.1

locale:
[1] LC_COLLATE=English_United States.utf8
[2] LC_CTYPE=English_United States.utf8
[3] LC_MONETARY=English_United States.utf8
[4] LC_NUMERIC=C
[5] LC_TIME=English_United States.utf8

time zone: America/New_York
tzcode source: internal

attached base packages:
[1] grid stats graphics grDevices utils datasets methods
[8] base

other attached packages:
[1] nnet_7.3-20 clue_0.3-68 poLCA_1.6.0.2
[4] MASS_7.3-65 scatterplot3d_0.3-45 survey_4.5
[7] survival_3.8-6 Matrix_1.7-5 matrixStats_1.5.0
[10] lubridate_1.9.5 forcats_1.0.1 stringr_1.6.0
[13] dplyr_1.2.1 purrr_1.2.2 readr_2.2.0
[16] tidyr_1.3.2 tibble_3.3.1 ggplot2_4.0.3
[19] tidyverse_2.0.0 gander_0.2.0 ellmer_0.4.1

loaded via a namespace (and not attached):
[1] gtable_0.3.6 xfun_0.57 httr2_1.2.2 lattice_0.22-9
[5] tzdb_0.5.0 vctrs_0.7.3 tools_4.6.0 generics_0.1.4
[9] parallel_4.6.0 cluster_2.1.8.2 pkgconfig_2.0.3 RColorBrewer_1.1-3
[13] S7_0.2.2 lifecycle_1.0.5 compiler_4.6.0 farver_2.1.2
[17] tinytex_0.59 mitools_2.4 htmltools_0.5.9 yaml_2.3.12
[21] crayon_1.5.3 pillar_1.11.1 tidyselect_1.2.1 digest_0.6.39
[25] stringi_1.8.7 labeling_0.4.3 splines_4.6.0 fastmap_1.2.0
[29] cli_3.6.6 magrittr_2.0.5 withr_3.0.2 scales_1.4.0
[33] rappdirs_0.3.4 bit64_4.8.2 timechange_0.4.0 rmarkdown_2.31
[37] bit_4.6.0 otel_0.2.0 hms_1.1.4 evaluate_1.0.5
[41] knitr_1.51 viridisLite_0.4.3 rlang_1.2.0 Rcpp_1.1.1-1.1
[45] glue_1.8.1 DBI_1.3.0 coro_1.1.0 rstudioapi_0.18.0
[49] vroom_1.7.1 jsonlite_2.0.0 R6_2.6.1